

<sub>1</sub> Mechanistic Explainability in Reinforcement  
<sub>2</sub> Learning

<sub>3</sub> Maxime Alaarabiou

<sub>4</sub> Saturday 26<sup>th</sup> September, 2026

# Chapter 1

## Clustering Agent Representations

As discussed in the previous chapter, current explainability methods for Reinforcement Learning (RL) still lack intrinsic approaches capable of explaining the internal mechanisms of deep neural policies. To the best of our knowledge, only [Zahavy et al., 2017] have proposed exploiting the Latent Space (LS) learned by neural policies in order to identify the concepts used by the agent. Its main idea is to analyze the Latent Representation (LR) produced by the Neural Network (NN) and organize them into clusters that correspond to high-level concepts.

While the core idea of this paper is closely aligned with our vision of explainability in RL, this approach suffers from two important limitations. First, the clusters are manually constructed. Such a procedure heavily depends on the user’s expertise and neither guarantees reproducibility or scales. Second, no evaluation protocol is proposed to assess the quality of the obtained clusters. Their relevance is essentially determined through visual inspection.

The Clustering Agent Representation (CAR) method proposed in this chapter addresses these two limitations. First, it automates concept discovery by relying on unsupervised clustering algorithms to identify the Concept Structure (CS) of the LS. Second, it introduces an objective evaluation metric based on the stability of the extracted representations across independently trained policies.

This chapter is organized as follows:

- Section 1.1 lays out the theoretical assumptions underlying the proposed CAR method, which we term the *Representation Convergence Hypothesis* (Hyp. 1).
- Section 1.2 then details the CAR method itself, covering both the extraction of LR using surrogate policies and the automatic detection of CS through clustering.
- Section 1.3 introduces the metrics we propose for evaluation.
- Section 1.4 situates CAR within the broader landscape of Mechanistic Explainability (ME).

## 1.1 Representation Convergence Hypothesis

The CAR method relies on a assumption, referred to as the *Representation Convergence Hypothesis* (Hyp. 1). This hypothesis follows the two assumptions established previously throughout this manuscript.

The *Mechanistic Hypothesis* (Hyp. ??) rejects the notion that deep NN constitute irreducible black boxes. Adopting a mechanistic perspective, we consider that every process results from a sequence of causal computations. Although these computations may be complex, they remain explainable through the analysis of the internal mechanisms.

The *Concept Hypothesis* (Hyp. ??) states that NN achieve generalization by learning concepts. During optimization, the network progressively organizes its LS into representations that capture the information required to solve the task.

Following these two hypotheses, it becomes possible to explain a NN by studying the organization of its LS. This family of approaches corresponds to ME (Definition ??). Since the CAR method is grounded in these two hypotheses, it falls within this field of research. More specifically, CAR focuses on a specific aspect of ME known as universality. Universality refers to the consistency with which a concept is recovered across different models trained on the same task [Templeton et al., 2024].

To provide a explanation for this phenomenon, we developed the *Representation Convergence Hypothesis* (Hyp. 1). This hypothesis states that, when NN are trained to solve the same task, a set of concepts should systematically emerge because these concepts provide a effective way of representing the problem. As a result, independently trained models should develop similar concepts in their learned representations.

### Hypothesis 1: Representation Convergence Hypothesis

For a given task, independently trained NN tend to develop similar sets of concepts because these concepts provide an efficient representation for solving the task.

The CAR method is designed to investigate the *Representation Convergence Hypothesis* (Hyp. 1) by assessing whether it can consistently recover similar concepts across independently trained models. The hypothesis therefore serves both as the foundation of the method and as the phenomenon it aims to investigate. If CAR consistently recovers the same set of concepts across different training runs, this set is more likely to reflect invariant properties of the task than model-specific artifacts. Conversely, the absence of representation convergence would provide evidence against the hypothesis and would also question the validity of the CAR approach.

It is important to clarify that this hypothesis does not imply convergence at the parameter level. Independently trained models are expected to converge toward different sets of weights. The proposed hypothesis concerns a abstract level of representation. We assume that the conceptual organization induced by their LR converges toward an equivalent CS, even though the numerical implementation of the networks differ.

The remainder of this chapter develops the methodology used to investigate this hypothesis.

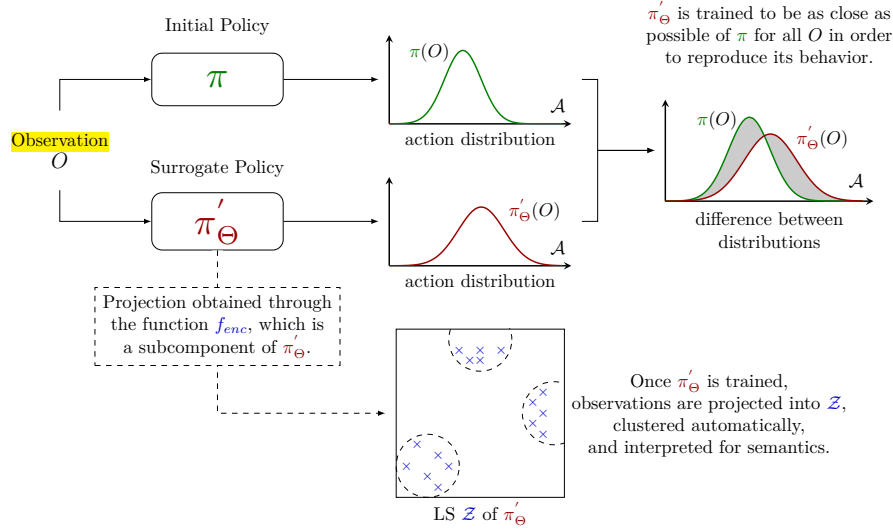


Figure 1.1: Illustration of the Clustering Agent Representation method.

## 1.2 Latent Clustering of Surrogate Policies

Following the work of [Zahavy et al., 2017], we analyze the LS through clustering techniques, placing our approach within the broader field of Deep Clustering. This design choice is motivated by our rejection of the *Superposition Hypothesis* (Hyp. ??) and, more generally, of any view that assumes a linearly organized CS, as discussed in Section ??.

The CAR methodology is composed of two main stages. The first stage trains multiple independent NN, called surrogate policies, which implement the same decision policy. The second stage clusters their LR in order to identify the underlying conceptual organization. Once these two stages have been completed, the resulting clustering can be analyzed from two complementary perspectives. First, the consistency of the resulting clusters across independent training runs can be assessed to determine whether the identified clusters are stable. Second, the semantic content of the clusters can be examined to determine whether they correspond to meaningful concepts. Figure 1.1 provides a visual overview of the two stages, from the imitation of the initial policy to the clustering of the surrogate representations.

This section is organized into three parts. The first explains why multiple surrogate policies are used to study the stability of the representations. The second describes how these policies are trained to match the initial policy as closely as possible. The third presents how their LR are clustered to identify the CS.

### 1.2.1 Multiple surrogate policies

The CAR method was developed to test the *Representation Convergence Hypothesis* (Hyp. 1). This requires training several NN independently and comparing their LR. Consistent similarities in the resulting CS would provide empirical support for this hypothesis.

In supervised learning, this protocol is straightforward since several models can be independently trained on the same target function. RL introduces an additional difficulty. Because of stochastic exploration, independently trained agents are not guaranteed to converge toward the same policy. Even if they achieve comparable performance, they may solve the task using different strategies. This makes it difficult to determine whether differences in the LS result from different representations of the same strategy or from differences in the learned strategies themselves.

To overcome this limitation, we consider an already trained initial policy. This policy is then used as a target for imitation learning [Ross et al., 2011]. In this process, several surrogate NN are independently trained in a supervised manner to reproduce its behavior. This makes it possible to investigate whether equivalent policies also exhibit equivalent conceptual organizations.

It is important to emphasize that the use of surrogate policies does not make CAR a surrogate explainability method, as defined in Subsection ???. The surrogate policies remain deep NN. Their role is not to simplify the decision process but to provide multiple independent realizations of the same policy function, enabling the study of representation convergence.

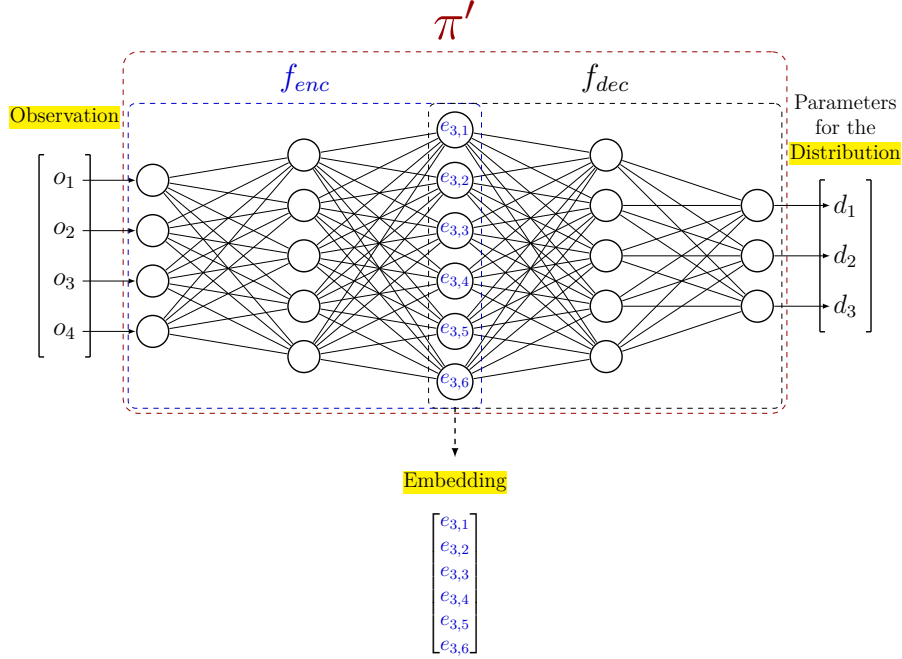
### 1.2.2 Training surrogate policies

Throughout the remainder of this manuscript, the initial policy we aim to explain is denoted as  $\pi$ . This initial policy is obtained without imposing any constraints on its origin. It may result from classical or deep reinforcement learning [Sutton and Barto, 2018], alpha-beta algorithm [Knuth and Moore, 1975], or even from demonstrations provided by humans [Ross and Bagnell, 2010]. The crucial requirement is the availability of a dataset containing a sufficient number of projections from the observation space  $\mathcal{O}$  to the action space  $\mathcal{A}$  induced by  $\pi$  to train a NN.

The replicated policies are referred to as surrogate policies. To assess the stability of the learned representations,  $n$  surrogate policies are trained independently, denoted by  $\pi'_1, \dots, \pi'_n$ . All surrogate policies share the same architecture. It consists of a NN with two components, an encoder  $f_{enc}$  and a decoder  $f_{dec}$ , highlighted by the blue and black dashed rectangles, respectively, in Figure 1.2. The encoder maps the observation  $O$  to the embedding, and the decoder reconstructs the parameters of the action distribution from this embedding. For a given surrogate policy  $\pi'_i$ , parameterized by  $\Theta_i$ , its encoder and decoder are written  $f_{enc_i}$  and  $f_{dec_i}$ .

To train each surrogate policy  $\pi'_i$  to accurately replicate the action distribution, we define a loss function  $\mathcal{L}$  optimized via gradient descent as introduced in Section ???. This loss, defined in Equation (1.1), is computed as the Mean Squared Error (MSE) between the parameters of the action distributions, obtained by applying the function  $p$ , which maps a distribution to its parameterization. In practice, this function  $p$  does not need to be explicitly defined, as neural policies directly output the parameters of the action distribution.

$$\min_{\Theta_i} \sum_{O \in \mathcal{O}} \text{MSE}\left(p(\pi(O)), p(\pi'_{\Theta_i}(O))\right) \quad (1.1)$$

Figure 1.2: Example of a surrogate policy architecture  $\pi'$ .

### 1.2.3 Cluster extraction

Once the surrogate policies are trained, their encoders are used to generate a point cloud from a set of observations. This point cloud provides a sample of the LS to be analyzed. The proposed CAR method adopts the deep clustering approach introduced in Section ???. Under this approach, conceptually similar observations are expected to produce nearby LR. A CS can therefore be viewed as a region of the LS containing related LR. To identify these regions, a clustering algorithm is applied to the point cloud.

The clustering is performed independently for each surrogate policy. For a surrogate policy  $\pi'_i$ , applying its encoder  $f_{enc_i}$  to a subset of observations  $\mathbb{O} \subseteq \mathcal{O}$  yields the set of embeddings  $\mathcal{E}_i$ . A clustering function  $f_{clust}$  then partitions the point cloud  $\mathcal{E}_i$  into groups of nearby LR, assigning each observation a cluster label and yielding the partition  $\mathcal{C}_i$ . The same observation set  $\mathbb{O}$  is used for every surrogate policy, so that the partitions  $\mathcal{C}_1, \dots, \mathcal{C}_n$  are defined over the same observations and can be compared. This comparison is the basis of the evaluation introduced in the following section.

Algorithm 1 states the complete CAR procedure, whose lines map directly onto the stages described above. Lines 1–2 prepare the imitation of the initial policy. They build the training dataset by pairing each observation with the parameters of the action distribution produced by  $\pi$ , and define the MSE loss of Equation (1.1). Lines 3–5 correspond to the first stage, in which each of the  $n$  surrogate policies is initialized randomly and trained independently by stochastic gradient descent. Lines 6–8 correspond to the second stage. The encoder of each surrogate policy embeds the shared observation set  $\mathbb{O}$  (Line 7), and the resulting

**Algorithm 1:** Clustering Agent Representation (CAR)**Input:**Initial policy  $\pi$ Observation dataset  $\mathbb{O}$ Number of surrogate policies  $n$ Surrogate policy architecture  $\pi'$  (encoder  $f_{enc}$ , decoder  $f_{dec}$ )Learning rate  $\eta$ Clustering function  $f_{clust}$ **Output:**Trained surrogate policies  $\pi'_{\Theta_1}, \dots, \pi'_{\Theta_n}$ Clusterings  $\mathcal{C}_1, \dots, \mathcal{C}_n$ *Behavioral cloning dataset built from the target policy  $\pi$* **1**  $\mathcal{D} \leftarrow \{(O, p(\pi(O))) \mid O \in \mathbb{O}\};$ *MSE loss between action-distribution parameters, as defined in Equation (1.1)***2**  $\mathcal{L} \leftarrow \text{MSE};$ *Independently train  $n$  surrogate policies by behavioral cloning***3** **for**  $i = 1, \dots, n$  **do****4**     Initialize  $\Theta_i$  randomly;*Behavioral cloning of  $\pi$  via SGD, see Algorithm ??***5**      $\Theta_i \leftarrow \text{SGD}(\mathcal{D}, \pi', \mathcal{L}, \eta, \Theta_i);$ *Cluster the latent representations of each surrogate policy***6** **for**  $i = 1, \dots, n$  **do***Embeddings produced by the encoder of  $\pi'_{\Theta_i}$* **7**      $\mathcal{E}_i \leftarrow \{f_{enc_i}(O) \mid O \in \mathbb{O}\};$ *Partition of the embedded observations into clusters***8**      $\mathcal{C}_i \leftarrow f_{clust}(\mathcal{E}_i);$ **9** **Return**  $(\pi'_{\Theta_1}, \dots, \pi'_{\Theta_n}), (\mathcal{C}_1, \dots, \mathcal{C}_n)$ 

173 point cloud is partitioned by  $f_{clust}$  (Line 8). Finally, Line 9 returns the surrogate  
 174 policies together with their partitions, which are the inputs of the evaluation.

175 Two choices govern this analysis without being part of the clustering algorithm  
 176 itself. The first is the choice of layer used to extract the representations. Each  
 177 hidden layer defines its own LS, reflecting the information represented at that  
 178 stage of the network. CAR can therefore be applied to the LR extracted from  
 179 any layer. This allows the conceptual organization of the policy to be examined  
 180 at different levels of the network.

181 The second is the number of clusters. CAR does not assume that a task  
 182 admits a single correct number of concepts. This choice instead determines the  
 183 granularity at which the conceptual organization is examined. A small number  
 184 reveals broad, strategic concepts, while a larger number reveals finer, tactical  
 185 ones. The influence of both choices is studied in Chapter 2.

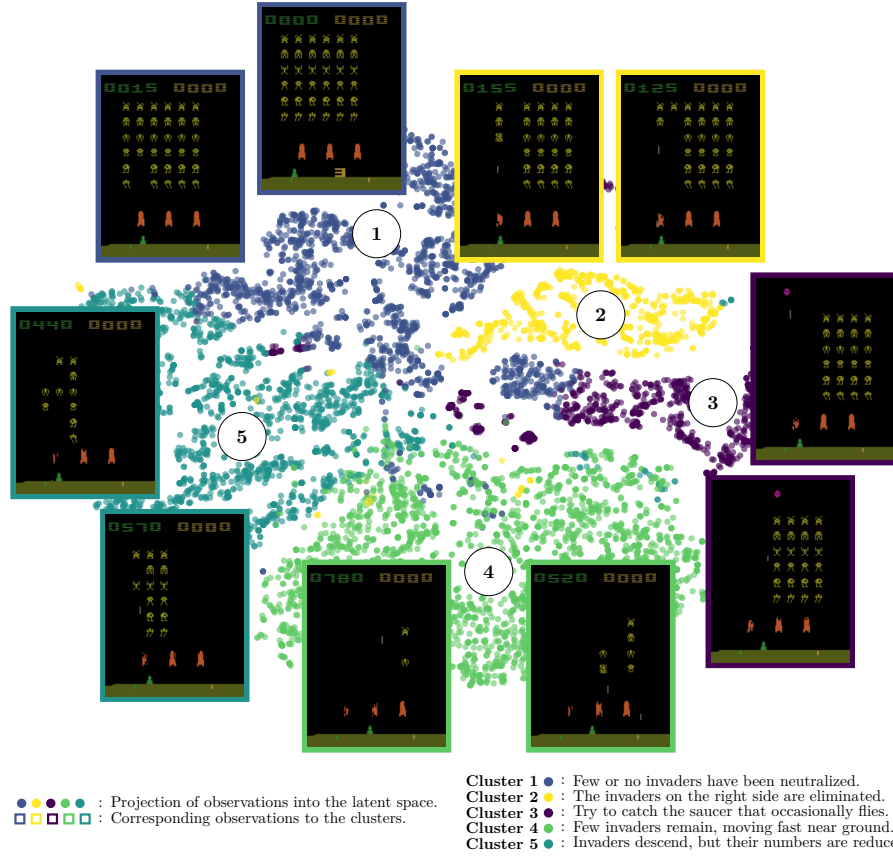


Figure 1.3: Two-dimensional projection of the LS embeddings of a surrogate policy trained on **Space Invaders**, together with the CAR clusters and their corresponding observations.

Figure 1.3 shows the outcome of this procedure for a surrogate policy trained on **Space Invaders**, using five clusters. The clusters are computed in the original high-dimensional LS. The point cloud is a two-dimensional projection used only for visualization. Each cluster gathers observations that correspond to a distinct, interpretable stage of a game:

- **Cluster 1:** The start of a game, with few or no invaders neutralized.
- **Cluster 2:** The right-hand group of invaders eliminated.
- **Cluster 3:** The pursuit of the bonus saucer.
- **Cluster 4:** The end of a game, with a few fast invaders close to the ground.
- **Cluster 5:** The middle of a game, with the invaders descending but already thinned out.

The position of an observation in the LS thus reflects the stage of the environment it belongs to, suggesting a correspondence between the geometry of the LS and the semantic properties of environment states.

This example illustrates what CAR is expected to produce, but it relies on the visual inspection of a single surrogate policy, which is precisely the type of evidence whose limitations motivated this work. Three aspects must therefore be evaluated before such an analysis can be trusted. First, the surrogate policy must faithfully reproduce the initial policy so that its explanations remain relevant to the target policy. Second, the recovered conceptual organization must be consistent across independently trained surrogate policies. Finally, the clusters must capture genuine abstractions rather than simply reproducing structure already present in the observations or action distributions.

The following section introduces one evaluation metric for each of these aspects and details how it is computed.

## 1.3 Evaluation Metrics

To evaluate the effectiveness of CAR, four evaluation metrics are introduced:

- ***Behavior Reconstruction*** : it measures how well a surrogate policy faithfully reproduces the behavior of the initial policy by comparing their cumulative rewards.
- ***Clustering Universality*** : it evaluates the stability of clusterings by measuring the similarity of clusters obtained from independently trained surrogate policies.
- ***Abstraction Score*** : it quantifies the extent to which LS clusters capture abstractions rather than directly reflecting the structure of observations, of the action distribution, or the network architecture prior to training.
- ***Adjusted Clustering Universality***: it combines the two previous metrics to measure the agreement between surrogate policies beyond what can be explained by the baselines used in *Abstraction Score*.

The remainder of this section details the motivation for each metric and describes how they are calculated. In the first subsection, we develop the *Behavior Reconstruction*. We then introduce the Adjusted Rand Index (ARI) measure, which serves as the foundation for the three other metrics introduced in the subsequent subsections, namely *Clustering Universality*, *Abstraction Score*, and *Adjusted Clustering Universality*.

### 1.3.1 Behavior Reconstruction

We introduce the *Behavior Reconstruction* metric to assess whether  $\pi'$  faithfully reproduces the behavior of  $\pi$ . This metric compares the returns obtained by the initial policy  $\pi$  and the surrogate policies  $\pi'$  over a set of evaluation episodes. We use the definition of return introduced in Section ??, where  $g(t, \gamma)$  denotes the discounted sum of rewards collected along a trajectory  $t$ . Here, the return is computed over complete episodes with  $\gamma = 1$ , so  $g$  corresponds to the total reward collected during the episode. Each policy is evaluated over  $m$  episodes. We denote by  $g_\pi^i$  the return obtained by the initial policy in episode  $i$ , and by  $g_{\pi'_j}^i$  the return obtained by surrogate policy  $\pi'_j$  in episode  $i$ . The *Behavior*

242 *Reconstruction* is then defined as the mean return of the surrogate policies  
 243 normalized by the mean return of the initial policy:

$$\text{Behavior Reconstruction} = \frac{1}{n \sum_{i=1}^m g_{\pi}^i} \sum_{j=1}^n \sum_{k=1}^m g_{\pi'_j}^k \quad (1.2)$$

244 In imitation learning, it is unlikely to expect the surrogate policy  $\pi'$  to  
 245 outperform the initial policy  $\pi$  it aims to approximate through supervised  
 246 learning [Ross and Bagnell, 2010]. Consequently this metric ranges in  $[0, 1]$ .  
 247 Values approaching 1 indicate minimal loss in reproducing  $\pi$ , while values close  
 248 to 0 indicate poor reproduction.

249 We deliberately adopt a reward-based definition rather than alternatives based  
 250 on actions or action distributions (e.g., training loss, accuracy, KL divergence  
 251 [Shlens, 2014], or optimal transport [Peyré and Cuturi, 2020]). Since surrogate  
 252 policies are trained in a supervised manner, they cannot develop strategies  
 253 beyond those demonstrated by the initial policy. This means that if the rewards  
 254 achieved are comparable, the surrogate policy is effectively copying the initial  
 255 policy’s behavior.

### 256 1.3.2 Adjusted Rand Index

257 Within CAR, concepts are assumed to correspond to clusters in the LS. Com-  
 258 bined with the *Representation Convergence Hypothesis* (Hyp. 1), this suggests  
 259 that independently trained surrogate policies should develop similar conceptual  
 260 organizations. Their LR should therefore produce similar clusterings when  
 261 analyzed using the same clustering procedure.

262 When a particular observation is processed by the network, it is expected to  
 263 activate the same concept regardless of the specific neural implementation of the  
 264 policy. If independently trained policies converge toward the same conceptual  
 265 organization, a given observation should activate the same concept across all  
 266 models. As a consequence, observations grouped together within a cluster for  
 267 one policy should also be grouped together for another independently trained  
 268 policy.

269 In the clustering literature, the most used measures for comparing two par-  
 270 titions is the ARI [Hubert and Arabie, 1985]. This measure quantifies the  
 271 agreement between cluster assignments independently of the spatial organiza-  
 272 tion of the clusters. Both metrics introduced later in this chapter, *Clustering*  
 273 *Universality* and *Abstraction Score*, rely on the ARI. Since the ARI is a complex  
 274 measure, we review its definition and discuss its main properties.

The ARI is derived from the Rand Index (RI) [Rand, 1971], which measures the agreement between two partitions by considering every possible pair of points in the point cloud. For each pair, four situations are possible:

- **Agreement:** The points belong to the same cluster in both partitions.
- **Agreement:** The points belong to different clusters in both partitions.
- **Disagreement:** The points belong to the same cluster in the first partition but to different clusters in the second.
- **Disagreement:** The points belong to different clusters in the first partition but to the same cluster in the second.

The RI measures the proportion of pairs for which the two partitions agree:

$$\text{RI} = \frac{\text{number of agreeing pairs}}{\text{total number of pairs}}.$$

Although the RI is intuitive, it suffers from an important limitation. In most clustering problems, the majority of pairs of points belong to different clusters. Consequently, even two completely independent partitions will agree on a large number of pairs, simply because both assign them to different clusters. This effect becomes more pronounced as the number of points or clusters increases.

The ARI addresses this issue by accounting for the agreement that would be expected purely by chance. It compares the observed agreement RI to the expected agreement under random partitions  $\mathbb{E}[\text{RI}]$ . The score is then normalized by the maximum agreement achievable beyond this random baseline,  $\max(\text{RI}) - \mathbb{E}[\text{RI}]$ . As a result, an ARI of 0 corresponds to the level of agreement expected from random clusterings, while an ARI of 1 indicates perfect agreement between the two partitions:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]}$$

In practice, the ARI is computed directly from the cluster assignments of the two partitions. Let clustering  $A$  be composed of partitions  $\{A_i\}$  and clustering  $B$  of partitions  $\{B_j\}$ . Let  $a_i = |A_i|$  denote the number of samples in partition  $i$  of clustering  $A$ , and  $b_j = |B_j|$  the number of samples in partition  $j$  of clustering  $B$ . Furthermore, let  $n_{ij} = |A_i \cap B_j|$  be the number of samples shared by partition  $i$  of clustering  $A$  and partition  $j$  of clustering  $B$ , and let  $n$  be the total number of samples. With these definitions, the ARI can be computed as:

$$\text{ARI} = \frac{\sum_{i,j} \binom{n_{ij}}{2} - \left[ \sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}{\frac{1}{2} \left[ \sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - \left[ \sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}$$

Because it is bounded by 1, a value such as 0.5 can be misleadingly interpreted as “halfway to a perfect match.” This interpretation is incorrect because the ARI is normalized against the agreement expected from two random partitions. As a result, values close to 1 require the two clusterings to be nearly identical, while a value of 0.5 already indicates substantial agreement [Amezquita et al., 2020].

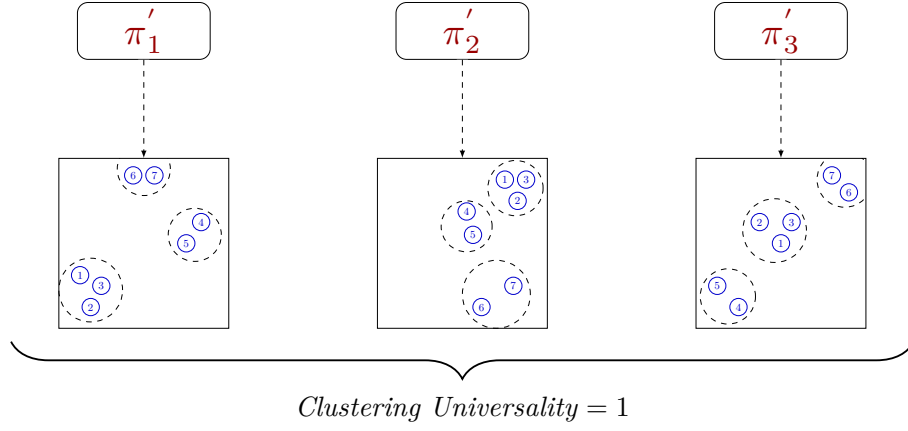


Figure 1.4: Illustration of *Clustering Universality* in the ideal case where projections of the same observations are grouped within the same clusters across all surrogate policies.

### 1.3.3 Clustering Universality

Having introduced the ARI, we can now define the *Clustering Universality* metric. While the ARI quantifies the similarity between two partitions, the objective of CAR is to assess the consistency of conceptual organizations across an entire set of independently trained surrogate policies.

To this end, the same set of observations is projected into the LS of each surrogate policy and clustered. Under the *Representation Convergence Hypothesis* (Hyp. 1), these independently obtained partitions are expected to be similar. *Clustering Universality* quantifies this by computing the mean pairwise ARI:

$$\text{Clustering Universality} = \frac{1}{\binom{n}{2}} \sum_{i=2}^n \sum_{j=1}^{i-1} \text{ARI}(\mathcal{C}_i, \mathcal{C}_j). \quad (1.3)$$

This metric takes values in  $[-1, 1]$ . A value close to 1 indicates that surrogate policies produce highly similar partitions, suggesting that the clusters reflect intrinsic properties of the policy. Conversely, a value near 0 implies that the similarity between partitions is no better than that of two randomly generated clusterings, meaning that the clustering fails to capture any meaningful structure in the policy. A negative value, on the other hand, signals that the agreement between partitions is worse than what would be expected by chance, indicating a systematic disagreement between the clusterings.

Figure 1.4 illustrates *Clustering Universality* in the ideal case. It shows three LS obtained by projecting the same seven observations, labeled 1 to 7, through three independently trained surrogate policies. Although the embeddings occupy different positions, the clusters group exactly the same elements in all three cases. The partitioning is therefore identical, resulting in an ARI of 1.

### 1.3.4 Abstraction Score

If *Clustering Universality* is high, it becomes important to determine what underlies this stability. Two possibilities arise:

- (i) the LS provides no abstraction, as defined in Definition ??, because it either reproduces the observation space or action distribution, or reflects a structure already present in the network architecture before training;
- (ii) the LS captures similarities in decision-making, resulting in behaviorally meaningful clusters.

To distinguish among these cases, we introduce the *Abstraction Score*. This metric contextualizes latent clusterings by examining how their patterns align with three reference baselines: the raw observations, the action distribution of the initial policy, and an untrained network of the same architecture.

We consider  $\mathcal{C}_{obs}$ , the clustering of the observation space  $\mathbb{O}$ , and  $\mathcal{C}_{act}$ , the clustering of the action distribution of the initial policy. Concretely,  $\mathcal{C}_{act}$  is computed on the parameters of this distribution (the logits, before the softmax) produced by the initial policy for the same observations, which are precisely the quantities the surrogate policies are trained to reproduce. We cluster these continuous parameters rather than the sampled discrete action, since a discrete action takes only a few distinct values, which would limit the number of clusters that can be distinguished. Both clusterings use the same algorithm, distance metric, and hyperparameters as those used for  $\mathcal{C}_i$  in the LS. This ensures that the three clusterings are obtained under the same conditions.

The third baseline addresses the case where the LS merely reflects an inductive bias of the network architecture rather than something learned. To this end, we introduce  $m$  untrained networks sharing the same architecture as the surrogate policies, but with randomly initialized, never-updated weights. Each of these  $m$  untrained networks is used to project the same observations into its own latent space, which is then clustered in the same way, yielding partitions  $\mathcal{C}_{untr,1}, \dots, \mathcal{C}_{untr,m}$ . This baseline isolates what the architecture already imposes on the representation before any learning takes place, so that a high similarity between  $\mathcal{C}_i$  and this baseline indicates that the corresponding structure cannot be attributed to training.

For each latent clustering  $\mathcal{C}_i$ , we define its similarity to these three baselines using the ARI:

$$\begin{aligned} A_i^{obs} &= \text{ARI}(\mathcal{C}_i, \mathcal{C}_{obs}) \\ A_i^{act} &= \text{ARI}(\mathcal{C}_i, \mathcal{C}_{act}) \\ A_i^{untr} &= \frac{1}{m} \sum_{k=1}^m \text{ARI}(\mathcal{C}_i, \mathcal{C}_{untr,k}) \end{aligned}$$

The *Abstraction Score* is subsequently calculated as one minus the average of the maximum absolute similarity values across these three reference baselines:

$$\text{Abstraction Score} = 1 - \frac{1}{n} \sum_{i=1}^n \max(|A_i^{obs}|, |A_i^{act}|, |A_i^{untr}|) \quad (1.4)$$

The *Abstraction Score* ranges from  $[0, 1]$ . A score around 0 means that the latent clusters are strongly aligned with the observation space, the action distribution, or the untrained network, reflecting no abstraction, case (i). A score close to 1 indicates that the latent clusters are independent of the observation space, the action distribution, and the network’s initialization, capturing abstractions that meaningfully reflect the agent’s behavior, case (ii).

### 1.3.5 Adjusted Clustering Universality

As discussed above, *Clustering Universality* alone does not indicate whether the recovered structure is meaningful. We therefore introduce the *Adjusted Clustering Universality*, which measures how much of the agreement between the different surrogate policies remains after accounting for the three trivial baselines.

$$\frac{\text{Adjusted Clustering Universality}}{\text{Clustering Universality}} = \frac{\text{Clustering Universality}}{\text{Clustering Universality}} - \frac{1}{n} \sum_{i=1}^n \max(|A_i^{obs}|, |A_i^{act}|, |A_i^{untr}|) \quad (1.5)$$

This metric ranges in  $[-1, 1]$ . A value close to 1 indicates that surrogate policies converge toward a partition that is both highly reproducible across seeds and clearly not explained by the observation space, the action distribution, or the network’s initialization, providing the strongest evidence for *Representation Convergence Hypothesis* (Hyp. 1). A value close to 0, or negative, indicates that whatever agreement exists between independently trained surrogate policies is largely trivial: the surrogate policies converge, but toward a partition that could just as easily emerge from raw pixel similarity, from the action distribution of the initial policy, or from an untrained network sharing the same architecture.

## 1.4 Conclusion

The CAR method was developed to investigate how concepts are organized within the representations of RL policies. It is based on three hypotheses introduced throughout this manuscript, the *Mechanistic Hypothesis* (Hyp. ??), the *Concept Hypothesis* (Hyp. ??), and the *Representation Convergence Hypothesis* (Hyp. 1).

CAR translates these assumptions into an empirical analysis. Multiple surrogate policies are trained independently to reproduce the same policy. Their LR are then clustered and compared to assess whether similar conceptual structures emerge across policies. The *Clustering Universality* and *Abstraction Score* provide quantitative measures for evaluating the consistency and abstraction of the resulting clusters.

This makes CAR a general procedure for analyzing agent representations. The analysis and evaluation metrics are fully automated and require no manual intervention or environment-specific adjustments. The method can therefore be applied to a wide range of RL settings and provides a basis for systematically studying the internal representations of neural policies.

The next chapter applies CAR to several environments to evaluate its ability to identify meaningful conceptual structures.

## Chapter 2

# Experiments

This chapter presents the experimental study of the Clustering Agent Representation (CAR) method introduced in the previous chapter. It focuses on evaluating the method’s ability to identify conceptual structures. The remainder of this chapter is organized as follows:

- Section 2.1 describes the software implementation of the CAR pipeline.
- Section 2.2 details the experimental configuration, covering the environments, the Reinforcement Learning (RL) setup, and the surrogate policy training.
- Section 2.3 presents the training of the initial and surrogate policies and evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy.
- Section 2.4 provides a quantitative analysis of the Latent Representations (LRs) using the *Clustering Universality* (CU), *Abstraction Score* (AS), and *Adjusted Clustering Universality* (ACU) metrics.
- Section 2.5 provides a qualitative analysis of the resulting clusters by manually examining their semantic content.
- Section 2.6 summarizes the findings and discusses their implications.

## 2.1 Software Implementation

This section details the experimental evaluation of the proposed method. Deep Learning (DL) and RL are strongly empirical fields, where experimental evaluation plays an important role in assessing how methods perform in practice. Particular attention is therefore given to the implementation of CAR. It relies on established libraries that provide efficient implementations of its main components. The code is made publicly available as open-source to support reproducible replication of the reported results.<sup>1</sup>

The CAR implementation is organized as a pipeline of five stages. Each stage performs a specific operation and produces the artifacts required by the

---

<sup>1</sup>[https://github.com/Maxalaar/Clustering\\_Agent\\_Latent\\_Space](https://github.com/Maxalaar/Clustering_Agent_Latent_Space)

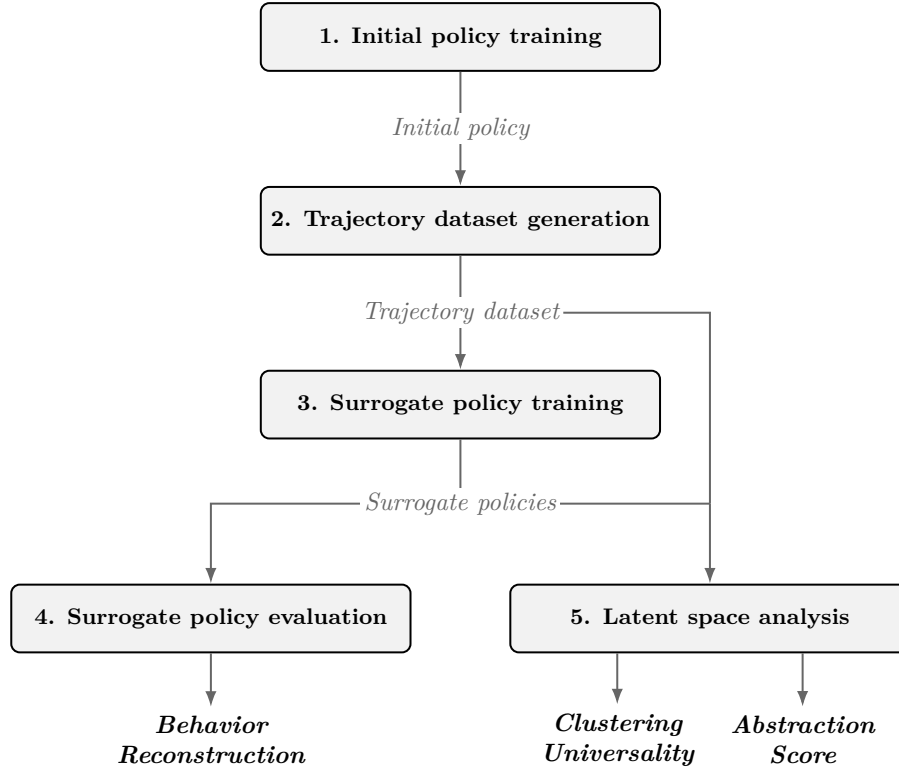


Figure 2.1: The CAR pipeline.

subsequent stages. The stages are implemented as independent executables allowing each stage to be rerun independently. The CAR pipeline is illustrated in Figure 2.1, and Table 2.1 summarizes the libraries used throughout the pipeline. The remainder of this section describes the purpose and implementation of each stage.

### 2.1.1 Initial policy training

The first stage trains the initial policy using RLlib [Moritz et al., 2018, Liang et al., 2018]. RLlib provides scalable implementations of RL algorithms and supports distributed training. The use of RLlib also allows CAR to be applied to environments implemented using the Gymnasium interface [Towers et al., 2024]. By default, the initial policy is trained using Proximal Policy Optimization (PPO) [Schulman et al., 2017]. Other RLlib algorithms, such as Asynchronous Proximal Policy Optimization (APPO) [Espeholt et al., 2018], Deep Q-Networks (DQN) [Mnih et al., 2013], and DreamerV3 [Hafner et al., 2023], can also be selected through the same configuration.

The policy and critic networks are implemented using PyTorch [Paszke et al., 2019]. PyTorch is the standard DL framework and is widely used for developing and training Neural Network (NN). It provides GPU acceleration and supports a wide range of NN architectures. This makes it well suited for implementing the different policy and critic architectures used in the experiments.

| Pipeline Stage                | Library                                         | Description                                                                       |
|-------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------------|
| Initial policy training       | RLlib [Moritz et al., 2018, Liang et al., 2018] | Distributed RL library used to train the initial policy.                          |
|                               | PyTorch [Paszke et al., 2019]                   | Deep learning framework implementing the policy and critic networks.              |
|                               | Gymnasium [Towers et al., 2024]                 | Common interface used to plug in any RL environment.                              |
| Trajectory dataset generation | RLlib [Moritz et al., 2018, Liang et al., 2018] | Distributed environment runners used to collect trajectories in parallel.         |
|                               | Apache Arrow [Apache Arrow, 2016]               | Columnar file format (Parquet) used to store the trajectory datasets.             |
| Surrogate policy training     | PyTorch Lightning [Falcon, 2019]                | Structured training loop used for the behavior cloning of the surrogate policies. |
| Surrogate policy evaluation   | RLlib [Moritz et al., 2018, Liang et al., 2018] | Wraps each surrogate policy as an RL policy to evaluate it in the environment.    |
| Latent space analysis         | cuML [RAPIDS, 2023]                             | GPU-accelerated clustering algorithms used to analyze the latent representations. |

Table 2.1: Main software libraries used at each stage of the CAR pipeline.

### 2.1.2 Trajectory dataset generation

The second stage builds the dataset used to train the surrogate policies. RLlib is used to collect trajectories in parallel using multiple environment runners. For each step, the dataset stores the observation together with the parameters of the action distribution produced by the initial policy.

Trajectories are stored as Parquet files [Apache Arrow, 2016]. Its columnar structure allows efficient data access and supports concurrent processing. This is useful here because the datasets contain around one million observation–action pairs and cannot be loaded entirely into memory.

### 2.1.3 Surrogate policy training

The third stage trains the surrogate policies using Behavioral Cloning (BC) on the trajectory dataset. This provides a supervised learning approach for replicating the initial policy. The training procedure is implemented using PyTorch Lightning [Falcon, 2019]. PyTorch Lightning provides a simplified interface for training NN with PyTorch. It handles common training operations such as data loading, optimization, checkpointing, and GPU execution.

### 2.1.4 Surrogate policy evaluation

The fourth stage evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy. Each surrogate is wrapped as an RLlib policy and evaluated in the original environment. Its mean cumulative reward over multiple episodes is compared with that of the initial policy. Evaluation episodes are collected using parallel environment runners.

This stage computes the *Behavior Reconstruction*. It therefore verifies that the surrogate policies reproduce the behavior of the initial policy sufficiently well before their LR’s are analyzed. If a surrogate does not meet the required level of behavior reconstruction, its training can be repeated.

### 2.1.5 Latent space analysis

The fifth stage performs the main analysis of CAR. The trajectory dataset and trained surrogate policies are used to generate the LRs. These representations form the point clouds that are analyzed using clustering algorithms.

The clustering step relies on the cuML library [RAPIDS, 2023], which provides GPU-accelerated clustering algorithms. This reduces the computational cost and allows point clouds containing tens of thousands of representations to be analyzed.

The resulting partitions are used to compute the *Clustering Universality* and the *Abstraction Score*. The stage also produces the material required for the qualitative analysis, including the clustered point clouds and the cluster assignment of each observation.

## 2.2 Experimental Configuration

This section describes the configuration of the experiments reported in this chapter. It covers the environments, the neural architectures used, and the main parameters of the pipeline.

### 2.2.1 Environments

All experiments presented in this manuscript use environments from the Atari benchmark [Bellemare et al., 2013], accessed through the Arcade Learning Environment. The use of Atari environments offers several advantages:

- **Standard benchmark:** Atari is widely used in the RL community, making the experimental results easier to compare with existing work.
- **Limited environment-specific bias:** The environments were not specifically designed for RL, reducing the risk of introducing biases related to their design.
- **Diversity:** The benchmark provides a diverse set of well-known environments with different types of tasks and levels of complexity, allowing CAR to be evaluated under different conditions.

The configuration of the Atari environments follows common practice in RL research [Machado et al., 2017]. In each environment, the agent receives visual observations in the form of raw pixels. Each image is resized to  $64 \times 64$  pixels and converted to grayscale to reduce memory requirements. To provide information about the dynamics of the environment, four consecutive observations are stacked along the channel dimension. The resulting observation has a shape of  $64 \times 64 \times 4$ . This allows the agent to infer the motion of objects from successive observations.

The actions correspond to the different buttons available on the Atari controller. The resulting action space is therefore discrete. Since Atari games are fully deterministic, sticky actions are introduced to add stochasticity to the trajectories. With a probability of 0.05, the environment repeats the action selected at the previous step instead of applying the new action. This prevents the agent from simply memorizing a fixed sequence of actions during training.

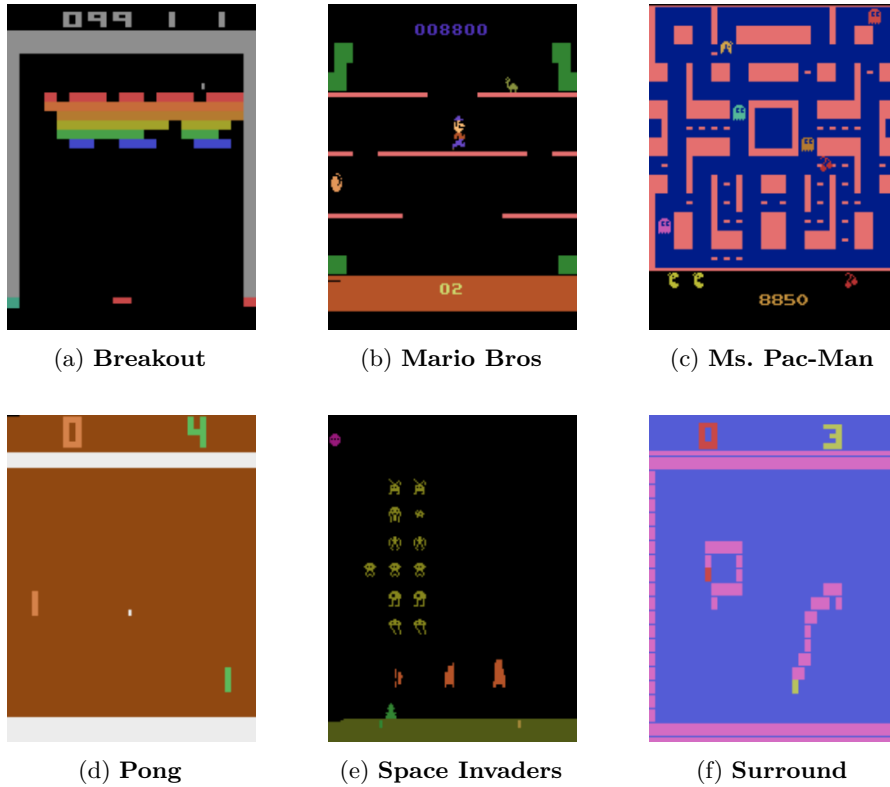


Figure 2.2: The six Atari environments used to evaluate CAR.

To reduce the duration of each episode, frame skipping is used. The agent selects a new action every five frames and repeats it for the following frames. This reduces the number of decisions required during an episode while maintaining sufficient temporal resolution for effective control.

The experiments use the six environments described below:

- **Breakout:** The agent moves a paddle at the bottom of the screen (Figure 2.2a) to bounce a ball into a wall of bricks. Each broken brick scores one point, and the ball speeds up as the wall is cleared.
- **Mario Bros.:** A single-screen platform game (Figure 2.2b) in which the player clears each phase by hitting enemies from below to flip them, then kicking them off the platforms before they recover. A central POW block can flip all enemies at once but can only be used a few times per phase.
- **Ms. Pac-Man:** The player moves through a maze (Figure 2.2c), eating pellets while avoiding four ghosts. Power pellets temporarily make the ghosts edible.
- **Pong:** A two-player game (Figure 2.2d) in which the agent controls one paddle and must send the ball past the opponent.
- **Space Invaders:** The agent moves a cannon along the bottom of the screen (Figure 2.2e), firing at descending rows of aliens while avoiding their

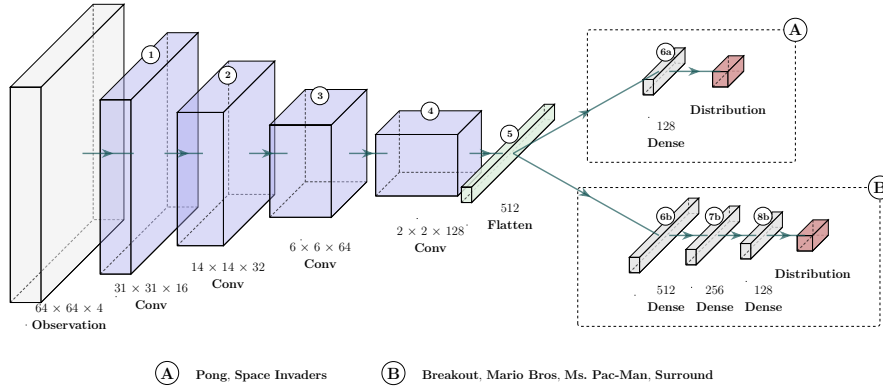


Figure 2.3: Architecture of the surrogate policies.

shots and using destructible shields. A bonus saucer occasionally crosses the top of the screen.

- **Surround:** A two-player game on a grid (Figure 2.2f) in which each player leaves a trail behind a moving cursor. The goal is to force the opponent to crash into a trail.

### 2.2.2 Neural architecture

The surrogate policies use a Convolutional Neural Network (CNN) to process the visual observations. The convolutional part consists of four layers with 16, 32, 64, and 128 channels, using  $4 \times 4$  kernels with a stride of 2. Each convolutional layer is followed by a LeakyReLU activation. The resulting representation is then flattened and passed through a MultiLayer Perceptron (MLP) that produces the parameters of the action distribution.

The architecture of the MLP varies across environments. For **Pong** and **Space Invaders**, it contains a single hidden layer with 128 units. For the other four environments, it contains three hidden layers with 512, 256, and 128 units. The size of the output layer matches the number of actions available in each environment: 4 for **Breakout**, 18 for **Mario Bros**, 9 for **Ms. Pac-Man**, 6 for **Pong**, 6 for **Space Invaders**, and 5 for **Surround**.

Figure 2.3 illustrates both configurations, together with the shape of the representation produced by each convolutional layer. Throughout the remainder of this chapter, the analyzed representations are numbered from 1 to 5. These representations correspond to the four convolutional layers and the flattened representation, which are common to all six environments.

The representations produced by the MLP are numbered separately. For **Pong** and **Space Invaders**, which use a single hidden layer, the representation is numbered 6a. For the other four environments, which use three hidden layers, the representations are numbered 6b, 7b, and 8b.

### 2.2.3 Other CAR parameters

Several other parameters must be defined for the different stages of the CAR pipeline. These parameters are less critical to the overall methodology than those discussed above, but they still need to be specified to reproduce the experiments.

| Environment configuration            |                      |
|--------------------------------------|----------------------|
| Frame size                           | $64 \times 64$       |
| Frames stacked per observation       | 4                    |
| Frame skip                           | 5                    |
| Sticky-action probability            | 0.05                 |
| Initial policy training              |                      |
| Algorithm                            | PPO                  |
| Optimizer                            | Adam                 |
| Learning rate                        | $1.5 \times 10^{-4}$ |
| GAE $\lambda$                        | 0.95                 |
| Policy clipping $\varepsilon$        | 0.1                  |
| Value-function clipping              | 10                   |
| Entropy coefficient                  | 0.01                 |
| KL coefficient                       | 0.5                  |
| Epochs per update                    | 10                   |
| Gradient clipping (global norm)      | 100                  |
| Surrogate policy training            |                      |
| Surrogate policies per environment   | 3                    |
| Observation–action pairs per dataset | $\geq 10^6$          |
| Batch size                           | 2048                 |
| Optimizer                            | Adam                 |
| Learning rate                        | $1 \times 10^{-4}$   |
| Latent space analysis                |                      |
| Embedding per surrogate policy       | 20 000               |
| Clustering algorithm                 | $k$ -means           |
| Distance                             | Euclidean distance   |
| Number of clusters $K$               | 2 to 20              |

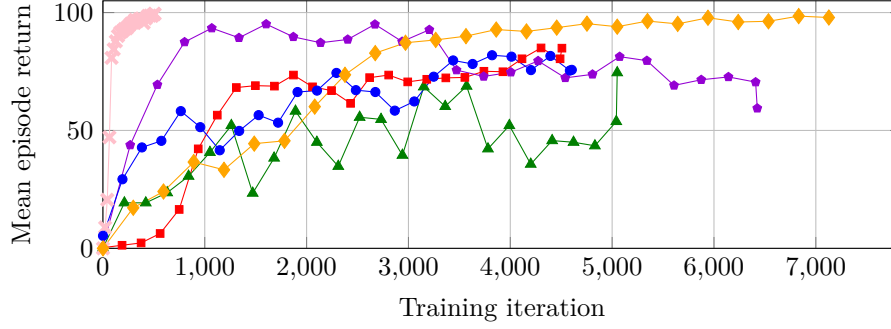
Table 2.2: Main experimental parameters.

Since each stage introduces its own configuration choices, there are too many parameters to discuss them all in detail. The main parameters used across the experiments are summarized in Table 2.2.

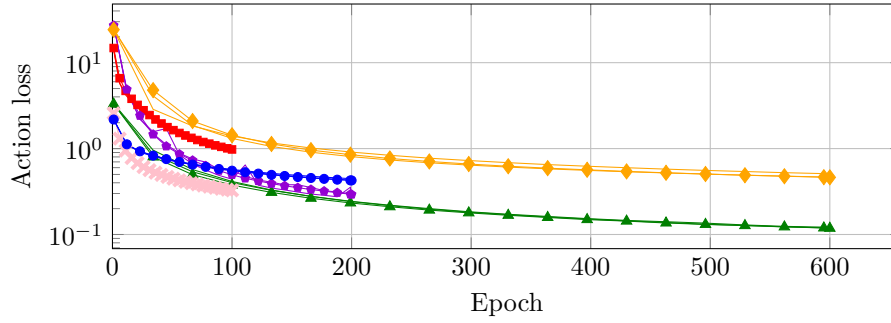
These settings are specific to the experiments presented in this chapter and do not restrict the CAR methodology itself. The different stages of the pipeline are modular and can be configured or replaced depending on the application. The evaluation metrics provide an objective basis for comparing these alternatives and assessing their effect on the consistency and meaningfulness of the resulting clusters.

## 2.3 Policy Performance and Surrogate Fidelity

This section reports the results of training the initial and surrogate policies for CAR, applied to the six environments. The first part verifies that the initial policies reach a competent level of play and evaluates how faithfully the surrogate policies reproduce their behavior using the *Behavior Reconstruction*.



(a) Training of the initial policies with PPO.



(b) Training action loss of the surrogate policies.

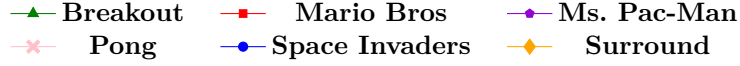


Figure 2.4: Training of the initial and surrogate policies.

### 2.3.1 Initial policy training

Figure 2.4a shows how the initial policies improve during PPO training. For each environment, the mean episode return is normalized so that 0% and 100% correspond to the lowest and highest mean return observed during that run. In every environment the return increases and then reaches a plateau, at which point training is stopped. The number of iterations required to reach this plateau varies from a few hundred, for **Pong**, to several thousand, for **Surround** and **Ms. Pac-Man**. For the subsequent analysis, the checkpoint with the best performance is retained rather than the final checkpoint.

To confirm that the trained policies play competently, their scores are compared with the reference scores reported by [Machado et al., 2017]. Each initial policy reaches a score of the same order as these references, so it constitutes a reasonable target for imitation.

### 2.3.2 Surrogate policy training

Figure 2.4b shows, for each environment, the training action loss of the three surrogate policies. In all cases, the loss decreases smoothly, and the three independently trained policies converge to the same value.

We further evaluate the ability of the surrogate policies to replicate the

| Environment           | <i>Behavior Reconstruction</i> |
|-----------------------|--------------------------------|
| <b>Breakout</b>       | 0.83 $\pm$ 0.03                |
| <b>Mario Bros</b>     | 0.99 $\pm$ 0.03                |
| <b>Ms. Pac-Man</b>    | 1.00 $\pm$ 0.03                |
| <b>Pong</b>           | 0.95 $\pm$ 0.01                |
| <b>Space Invaders</b> | 0.97 $\pm$ 0.07                |
| <b>Surround</b>       | 1.00 $\pm$ 0.01                |

Table 2.3: Evaluation of *Behavior Reconstruction* across the six Atari environments.

behavior of the initial policies using the *Behavior Reconstruction* (Table 2.3). The surrogate policies reach a *Behavior Reconstruction* of 0.96 on average over 100 evaluation episodes, ranging from 0.83 for **Breakout** to 1.00 for **Ms. Pac-Man** and **Surround**. They therefore reproduce the decisions of the initial policies closely enough for their LR to be analyzed.

## 2.4 Clustering Quality of the Latent Space

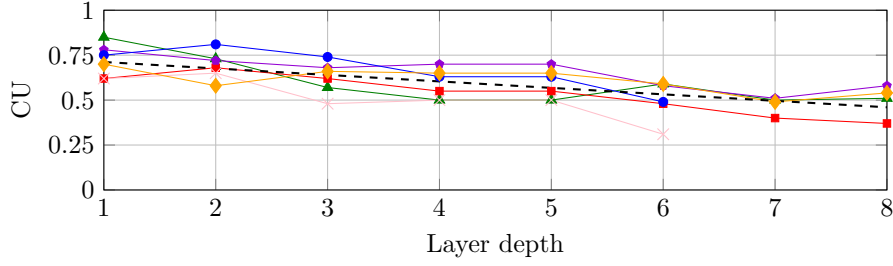
This section analyzes the Latent Space (LS) of the surrogate policies using CU, AS, and ACU. To do so, a random subset of 20 000 observations is selected for each environment. This subset is then embedded into the LS of each surrogate policy at each analyzed layer. This produces a point cloud for each combination of environment, surrogate policy, and layer. Each point cloud is partitioned using *k*-means clustering [MacQueen, 1967] for each number of clusters from 2 to 20.

The three metrics are then computed from these point clouds and their corresponding clusterings. The full results across all environments and analyzed layers are reported in Appendix ???. Two complementary analyses are then conducted on these results. First, the evolution of the three metrics across the different layers is examined to study the effect of representation depth. Second, for each environment, the layer achieving the highest average ACU is selected and analyzed in more detail across the different numbers of clusters.

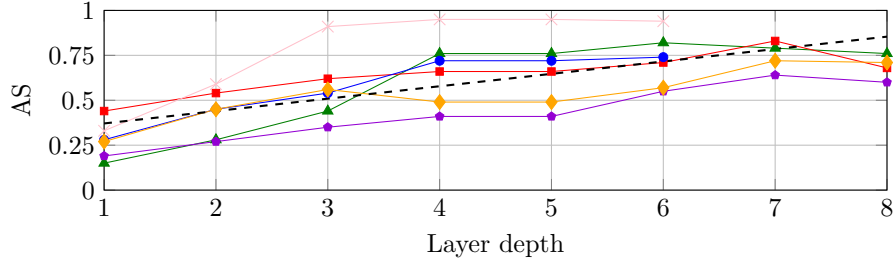
### 2.4.1 Analysis across layers

The first analysis examines how the three metrics evolve with the depth of the analyzed layer. For each layer, CU, AS, and ACU are averaged over the tested cluster numbers. Figure 2.5 reports these averages as a function of layer depth.

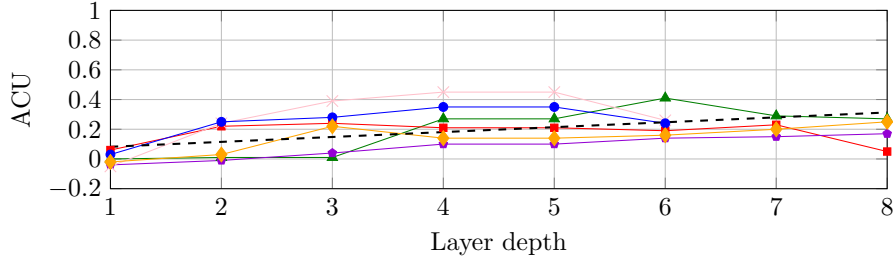
Across environments, CU tends to be highest in the earliest layers and decreases with depth. AS follows the opposite trend and generally increases as the representation becomes less tied to the raw observation, the action distribution, and the network’s initialization. This increase is however not monotonic in the deepest layers. For **Mario Bros** and **Ms. Pac-Man**, AS decreases at the last layer, from 0.83 to 0.68 and from 0.64 to 0.60 respectively, as the latent clustering becomes closer to the clustering of the action distribution, this layer being directly followed by the output layer. As ACU combines these two effects, its evolution depends on the environment.



(a) CU with respect to the layer depth.



(b) AS with respect to the layer depth.



(c) ACU with respect to the layer depth.



Figure 2.5: CU, AS, and ACU averaged over cluster numbers as a function of layer depth.

The corresponding linear fits show that CU decreases from 0.72 at the first layer to 0.50 at the last, with a slope of approximately  $-0.04$  per layer, while AS increases from 0.28 to 0.69, with a slope of approximately  $+0.06$  per layer. Over the same range, ACU increases more moderately from 0.00 to 0.19, with a slope of approximately  $+0.03$  per layer. This suggests that, despite the environment-specific variations, deeper representations tend on average to provide a better trade-off between CU and AS.

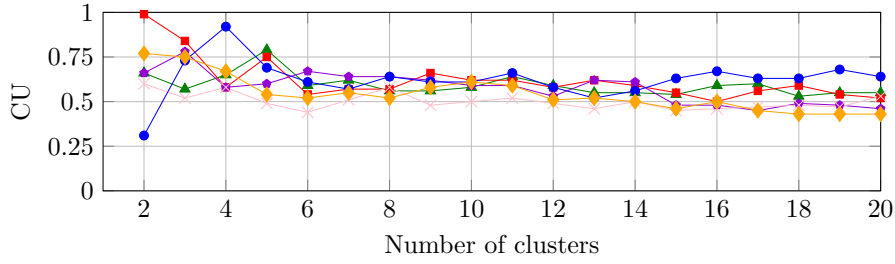
### 2.4.2 Analysis of the selected layers

The second analysis examines the effect of the number of clusters on the three metrics. For each environment, the layer with the highest average ACU across the tested cluster numbers is selected. Table 2.4 reports the selected layer,

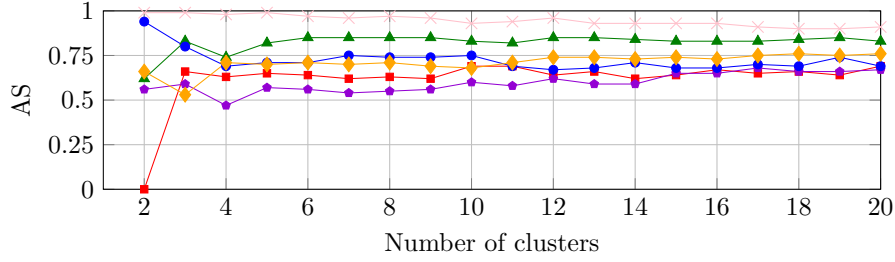
together with the corresponding average CU, AS, and ACU.

Figure 2.6 reports the evolution of CU, AS, and ACU as the number of clusters varies. **Pong** achieves the highest average ACU among the six environments, with a value of  $0.45 \pm 0.06$ . This results from a moderate CU combined with an AS that saturates from the third layer onward. In contrast, **Ms. Pac-Man** achieves the lowest AS, with  $0.60 \pm 0.05$ . The resulting ACU,  $0.17 \pm 0.06$ , is the lowest across the six environments.

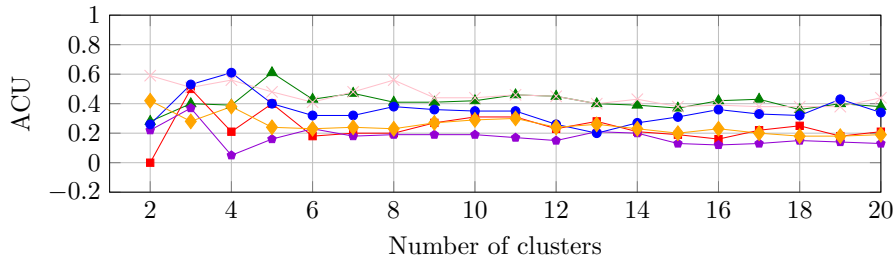
The quantitative analysis provides a measure of the consistency and abstraction of the identified clusters, but does not directly reveal their semantic content. The following section therefore provides a qualitative analysis of the clusters to examine the concepts they represent.



(a) CU with respect to the number of clusters.



(b) AS with respect to the number of clusters.



(c) ACU with respect to the number of clusters.

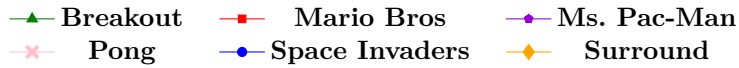


Figure 2.6: CU, AS, and ACU with respect to the number of clusters, for each of the six environments, for the layer that maximizes the average ACU in that environment.

| Environment    | Layer | CU              | AS              | ACU             |
|----------------|-------|-----------------|-----------------|-----------------|
| Breakout       | 6b    | 0.59 $\pm$ 0.06 | 0.82 $\pm$ 0.05 | 0.41 $\pm$ 0.06 |
| Mario Bros     | 3     | 0.62 $\pm$ 0.12 | 0.62 $\pm$ 0.15 | 0.24 $\pm$ 0.10 |
| Ms. Pac-Man    | 8b    | 0.58 $\pm$ 0.08 | 0.60 $\pm$ 0.05 | 0.17 $\pm$ 0.06 |
| Pong           | 4     | 0.50 $\pm$ 0.04 | 0.95 $\pm$ 0.03 | 0.45 $\pm$ 0.06 |
| Space Invaders | 4     | 0.63 $\pm$ 0.11 | 0.73 $\pm$ 0.06 | 0.35 $\pm$ 0.09 |
| Surround       | 8b    | 0.54 $\pm$ 0.10 | 0.71 $\pm$ 0.05 | 0.25 $\pm$ 0.06 |

Table 2.4: Layer that maximizes the average ACU across  $K \in [2, 20]$  for each environment, together with the corresponding average CU, AS, and ACU over  $K$ .

## 2.5 Semantic Interpretation of the Clusters

The metrics reported in Section 2.4 evaluate the LS clusterings quantitatively but do not indicate what the resulting clusters actually represent. This section complements them with a qualitative, hand-labeled inspection of the clusters obtained for each environment.

For each environment, two clusterings are selected for the qualitative analysis. They are obtained using two different numbers of clusters and may correspond to different layers of the network. The choice of the layer and the number of clusters is based on the following criteria:

- **High ACU:** The selected clustering should achieve a sufficiently high ACU, indicating a good balance between consistency and abstraction.
- **Manual analysis:** The number of clusters must remain small enough to allow the resulting clusters to be examined manually.
- **Limited use of two-cluster partitions:** Clusterings with two clusters often achieve a high ACU, but provide limited information about the organization of the LS. Therefore, at most one of the two selected clusterings uses two clusters, while the other uses a larger number of clusters to provide a more detailed view of the LS.

### 2.5.1 Breakout

Two clusterings are examined for **Breakout** (Table 2.5), obtained at layer 4 with  $K = 2$  and at layer 6b with  $K = 5$ .

At layer 4 (Figure 2.7), the partition captures the position of the ball relative to the brick wall. The ball remains below the wall in cluster A. The ball has passed behind the wall in cluster B, allowing it to continue clearing bricks without further interaction with the paddle.

At layer 6b (Figure 2.8), the partition again isolates situations in which the ball has passed behind the wall in cluster E. It also identifies the digging of a tunnel through the left side of the wall in cluster C. Finally, it distinguishes different paddle positions at the beginning of the episode in clusters D, F, and G.

The clustering at layer 4 with  $K = 2$  is almost perfectly reproducible across surrogate policies, with a CU of 0.99 and an ACU of 0.87. The clustering at layer 6b with  $K = 5$  is less consistent, with a CU of 0.79 and an ACU of 0.61, but reveals additional information about the organization of the LS.

| Layer 4  | $K = 2$                                             | CU 0.99 | AS 0.88 | ACU 0.87 |
|----------|-----------------------------------------------------|---------|---------|----------|
| A        | The ball is below the brick wall.                   |         |         |          |
| B        | The ball passes behind the brick wall.              |         |         |          |
| Layer 6b | $K = 5$                                             | CU 0.79 | AS 0.82 | ACU 0.61 |
| C        | Digging a tunnel through the left side of the wall. |         |         |          |
| D        | Start of the episode, paddle on the right.          |         |         |          |
| E        | The ball is behind the brick wall.                  |         |         |          |
| F        | Start of the episode, paddle in the middle.         |         |         |          |
| G        | Start of the episode, paddle on the left.           |         |         |          |

Table 2.5: CU, AS, and ACU of the two clusterings analyzed in the **Breakout** environment.

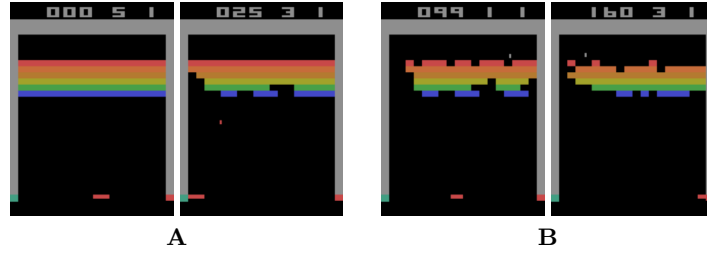


Figure 2.7: Clusters A–B identified by CAR in the **Breakout** environment (layer 4,  $K = 2$ ).

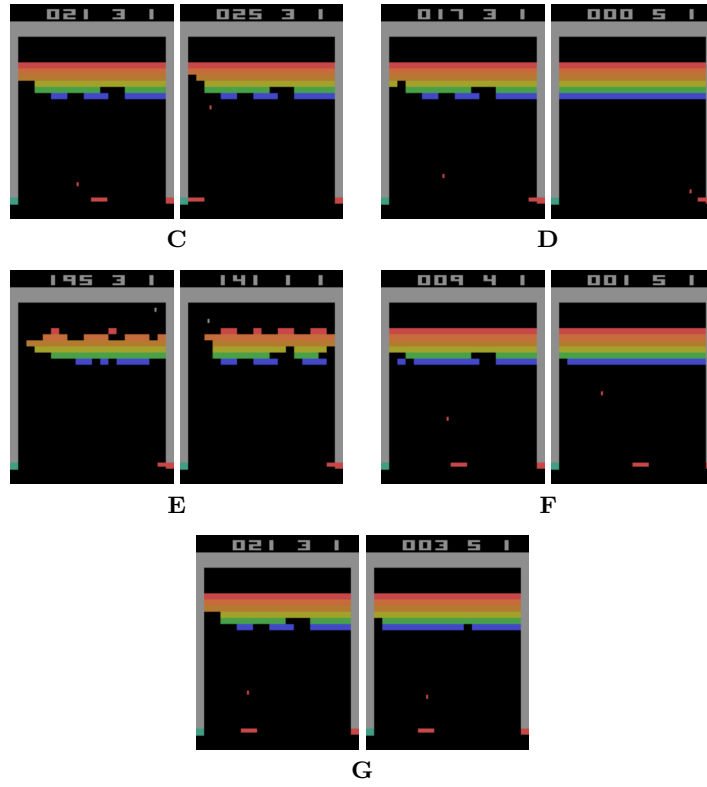


Figure 2.8: Clusters C–G identified by CAR in the **Breakout** environment (layer 6b,  $K = 5$ ).

### 2.5.2 Mario Bros

Two clusterings are examined for **Mario Bros** (Table 2.6), both obtained at layer 3, with  $K = 3$  and  $K = 5$ .

With  $K = 3$  (Figure 2.9), the partition is organized around the location of the action. Mario fights enemies on the left side of the screen in cluster A. He fights enemies on the right side of the screen in cluster C. The POW block is present in cluster B, isolating a fixed and recognizable element of the level.

With  $K = 5$  (Figure 2.10), the partition provides a more detailed description of the situations occurring in the level. Mario is on the top-left platform in cluster D. Mario attacks enemies at the top left in cluster F. Mario hits enemies from below in cluster H. Mario simply moves through the level in cluster G. The POW block remains isolated in cluster E.

Increasing  $K$  from 3 to 5 lowers CU from 0.84 to 0.75. The AS remains nearly unchanged, with values of 0.66 and 0.65. This results in a decrease in ACU from 0.50 to 0.40.

| Layer 3 | $K = 3$                                               | CU 0.84 | AS 0.66 | ACU 0.50 |
|---------|-------------------------------------------------------|---------|---------|----------|
| A       | Mario fights enemies on the left side of the screen.  |         |         |          |
| B       | The POW block is present.                             |         |         |          |
| C       | Mario fights enemies on the right side of the screen. |         |         |          |
| Layer 3 | $K = 5$                                               | CU 0.75 | AS 0.65 | ACU 0.40 |
| D       | Mario is on the top-left platform.                    |         |         |          |
| E       | The POW block is present.                             |         |         |          |
| F       | Mario attacks the enemies at the top right.           |         |         |          |
| G       | Mario moves through the level.                        |         |         |          |
| H       | Mario hits enemies from below on the left side.       |         |         |          |

Table 2.6: CU, AS, and ACU of the two clusterings analyzed in the **Mario Bros** environment.

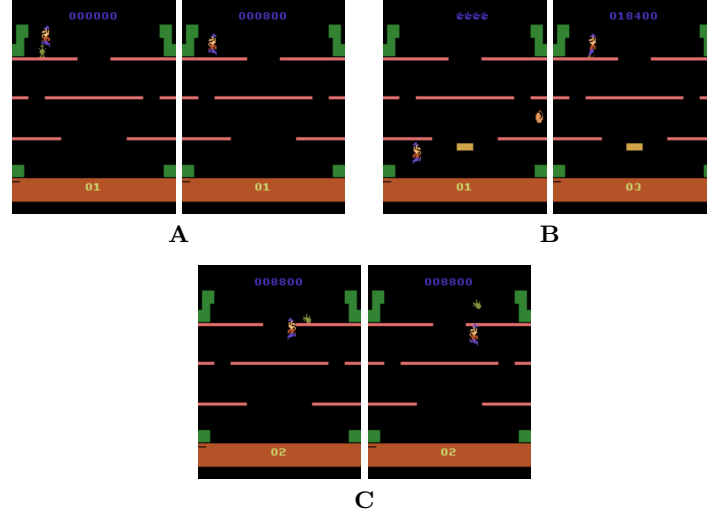


Figure 2.9: Clusters A–C identified by CAR in the **Mario Bros** environment (layer 3,  $K = 3$ ).

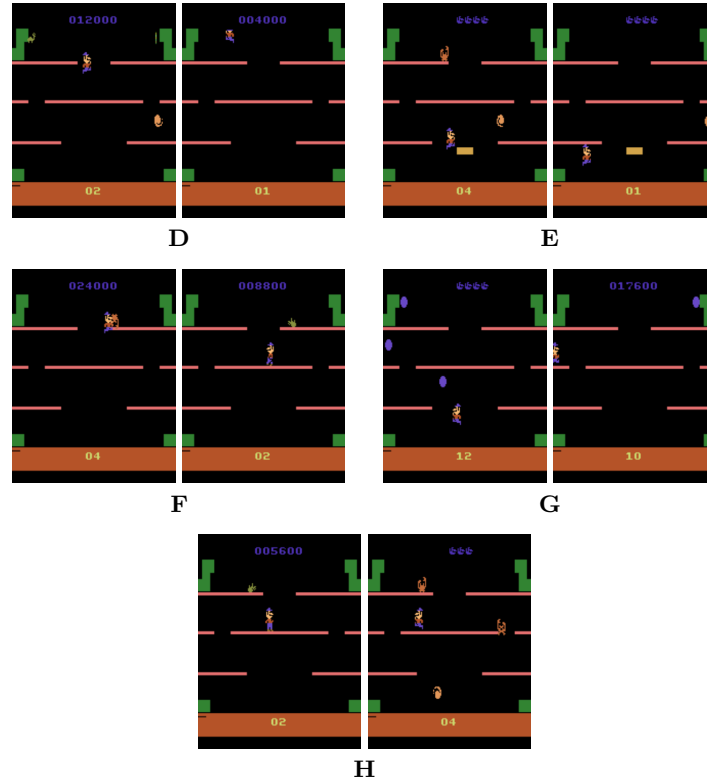


Figure 2.10: Clusters D–H identified by CAR in the **Mario Bros** environment (layer 3,  $K = 5$ ).

### 2.5.3 Ms. Pac-Man

Two clusterings are examined for **Ms. Pac-Man** (Table 2.7), obtained at layer 6b with  $K = 3$  and at layer 4 with  $K = 5$ .

With  $K = 3$  (Figure 2.11), the partition captures the progress and state of the game rather than a specific spatial feature of the observation. The start of the episode is captured in cluster B. The middle of the episode is captured in cluster A. The end of the episode is captured in cluster C.

With  $K = 5$  (Figure 2.12), the partition provides a more detailed description of the episode. The initial animation, during which the character cannot move, is isolated in cluster H. The beginning of actual play is captured in cluster F. A situation specific to the game mechanics, in which Pac-Man is in booster mode and waits in front of the ghost spawn to eat the ghosts, is isolated in cluster D. The end of the episode, when at most one power pellet remains, is separated into two similar clusters, E and G.

The two clusterings have the same CU of 0.88. The clustering at layer 4 has a lower AS of 0.47 compared with 0.52 at layer 6b. This results in a lower ACU of 0.35 compared with 0.40.

| Layer 6b | $K = 3$                                                                             | CU 0.88 | AS 0.52 | ACU 0.40 |
|----------|-------------------------------------------------------------------------------------|---------|---------|----------|
| A        | Middle of the episode.                                                              |         |         |          |
| B        | Start of the episode.                                                               |         |         |          |
| C        | End of the episode.                                                                 |         |         |          |
| Layer 4  | $K = 5$                                                                             | CU 0.88 | AS 0.47 | ACU 0.35 |
| D        | Pac-Man is in booster mode and waits in front of the ghost spawn to eat the ghosts. |         |         |          |
| E        | End of the episode, at most one power pellet remains (similar to cluster G).        |         |         |          |
| F        | Start of the episode.                                                               |         |         |          |
| G        | End of the episode, at most one power pellet remains (similar to cluster E).        |         |         |          |
| H        | Start-of-episode animation, the character cannot move.                              |         |         |          |

Table 2.7: CU, AS, and ACU of the two clusterings analyzed in the **Ms. Pac-Man** environment.

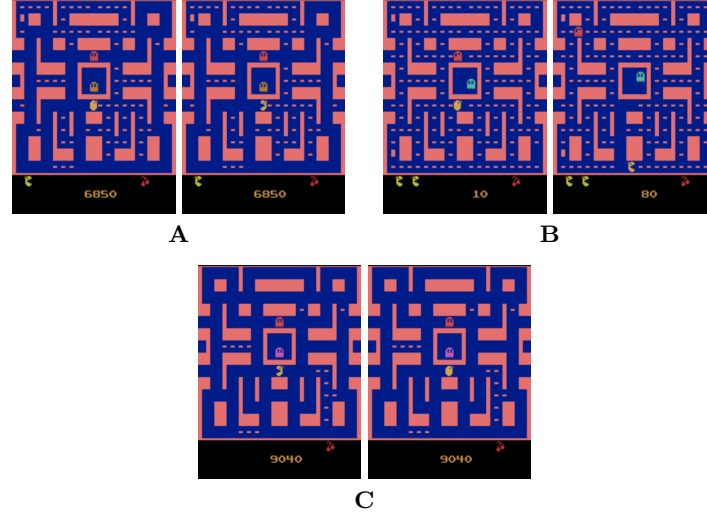


Figure 2.11: Clusters A–C identified by CAR in the **Ms. Pac-Man** environment (layer 6b,  $K = 3$ ).

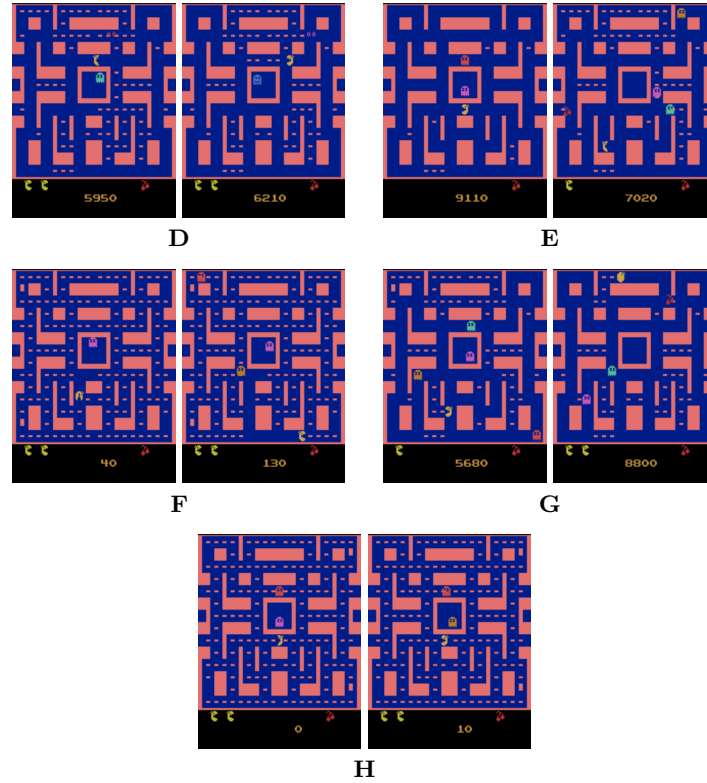


Figure 2.12: Clusters D–H identified by CAR in the **Ms. Pac-Man** environment (layer 4,  $K = 5$ ).

#### 2.5.4 Pong

Two clusterings are examined for **Pong** (Table 2.8), obtained at layer 2 with  $K = 2$  and at layer 3 with  $K = 3$ .

At layer 2 (Figure 2.13), the partition separates the observations according to the number of digits in the player’s score. A one-digit score corresponds to cluster A. A two-digit score corresponds to cluster B. This reflects the progress of the game, but only through its visual display at the top of the screen. The partition therefore captures a feature of the observation that provides no information about how the agent plays.

At layer 3 (Figure 2.14), the partition instead follows the vertical position of the ball. The ball is at the top of the field in cluster C. It is in the middle of the field in cluster D. It is at the bottom of the field in cluster E. This feature directly determines where the agent needs to place its paddle.

The clustering at layer 2 is perfectly reproducible and perfectly abstract, with CU, AS, and ACU all equal to 1.00. The clustering at layer 3 is less consistent, with a CU of 0.67, and reaches a lower ACU of 0.65.

| Layer 2 | $K = 2$                            | CU 1.00 | AS 1.00 | ACU 1.00 |
|---------|------------------------------------|---------|---------|----------|
| A       | The player’s score has two digits. |         |         |          |
| B       | The player’s score has one digit.  |         |         |          |
| Layer 3 | $K = 3$                            | CU 0.67 | AS 0.98 | ACU 0.65 |
| C       | The ball is at the top.            |         |         |          |
| D       | The ball is in the middle.         |         |         |          |
| E       | The ball is at the bottom.         |         |         |          |

Table 2.8: CU, AS, and ACU of the two clusterings analyzed in the **Pong** environment.

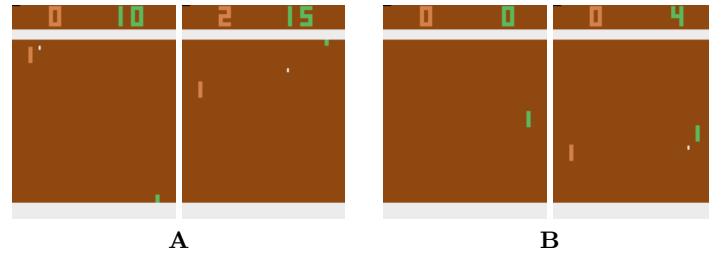


Figure 2.13: Clusters A–B identified by CAR in the **Pong** environment (layer 2,  $K = 2$ ).

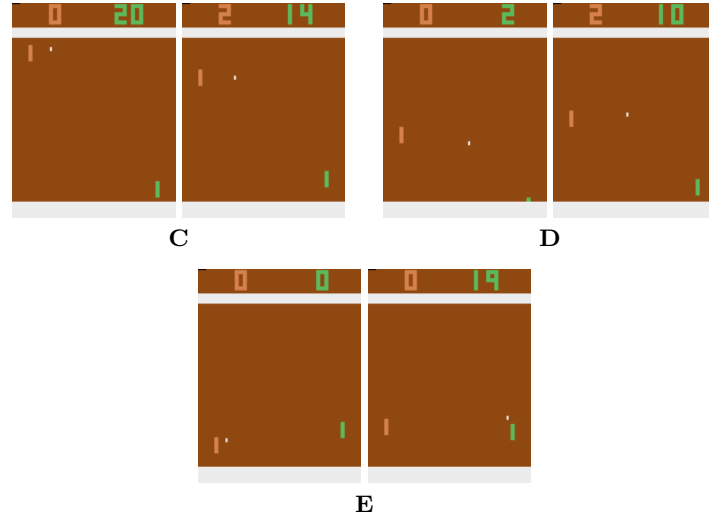


Figure 2.14: Clusters C–E identified by CAR in the **Pong** environment (layer 3,  $K = 3$ ).

### 2.5.5 Space Invaders

Two clusterings are examined for **Space Invaders** (Table 2.9), obtained at layer 4 with  $K = 4$  and at layer 2 with  $K = 5$ .

At layer 4 (Figure 2.15), the partition tracks the progression of the episode. The start of the episode is captured in cluster D. The middle of the episode is captured in cluster B. The end of the episode is captured in cluster C. A specific behavior in which the agent has cleared the leftmost column and is positioned to catch the flying saucer that occasionally crosses the top of the screen is captured in cluster A.

At layer 2 (Figure 2.16), the partition again tracks the progression of the episode. The start of the episode is captured in cluster G. The middle of the episode is captured in cluster F. The end of the episode is captured in cluster I. Attacking the first two columns of the formation is captured in cluster E. Trying to catch the flying saucer is captured in cluster H.

The clustering at layer 4 has a CU of 0.92 and a AS of 0.69, resulting in a ACU of 0.61. The clustering at layer 2 has the same CU of 0.92, but a lower AS of 0.49, resulting in a lower ACU of 0.41.

| Layer 4 | $K = 4$                                                                   | CU 0.92 | AS 0.69 | ACU 0.61 |
|---------|---------------------------------------------------------------------------|---------|---------|----------|
| A       | The left column has been cleared; the flying saucer occasionally appears. |         |         |          |
| B       | Middle of the episode.                                                    |         |         |          |
| C       | End of the episode.                                                       |         |         |          |
| D       | Start of the episode.                                                     |         |         |          |
| Layer 2 | $K = 5$                                                                   | CU 0.92 | AS 0.49 | ACU 0.41 |
| E       | Attacks the first two columns of the formation.                           |         |         |          |
| F       | Middle of the episode.                                                    |         |         |          |
| G       | Start of the episode.                                                     |         |         |          |
| H       | Tries to catch the flying saucer.                                         |         |         |          |
| I       | End of the episode.                                                       |         |         |          |

Table 2.9: CU, AS, and ACU of the two clusterings analyzed in the **Space Invaders** environment.

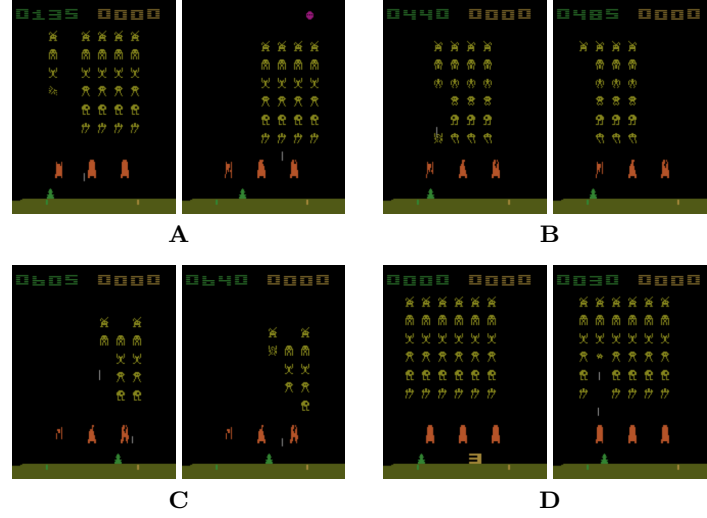


Figure 2.15: Clusters A–D identified by CAR in the **Space Invaders** environment (layer 4,  $K = 4$ ).

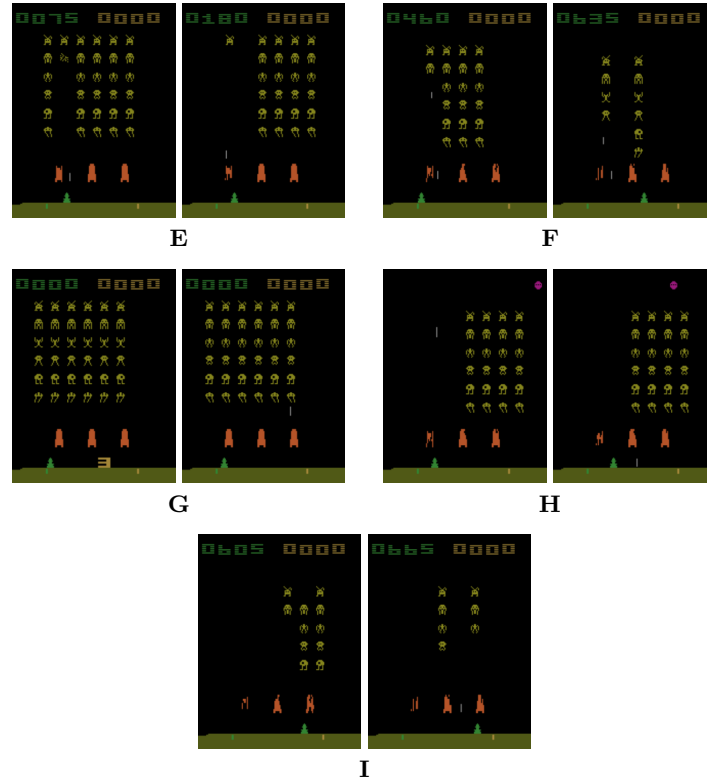


Figure 2.16: Clusters E–I identified by CAR in the **Space Invaders** environment (layer 2,  $K = 5$ ).

### 2.5.6 Surround

Two clusterings are examined for **Surround** (Table 2.10), obtained at layer 8b with  $K = 4$  and at layer 7b with  $K = 6$ .

At layer 8b (Figure 2.17), the partition captures the start of the episode in cluster B. The outcome of the episode, when the opponent loses, is captured in cluster D. The player's trail is confined to the bottom-left area in cluster A. The player reaches the top of the level from the left and then moves along the top in cluster C.

At layer 7b (Figure 2.18), the partition again captures the start of the episode in cluster J. The outcome of the episode, when the opponent loses, is captured in cluster F. The player moves in the upper-left corner in cluster E. The player moves in the lower-left corner in cluster G. The end of the game is played in the lower-left corner in cluster H. The player crosses the level from the top left to the bottom right in cluster I.

The clustering at layer 8b has a CU of 0.67, a AS of 0.71, and a ACU of 0.38. The clustering at layer 7b reaches the same values, with a CU of 0.67, a AS of 0.71, and a ACU of 0.38.

| Layer 8b | $K = 4$                                                             | CU 0.67 | AS 0.71 | ACU 0.38 |
|----------|---------------------------------------------------------------------|---------|---------|----------|
| A        | The agent's trail remains confined to the bottom-left area.         |         |         |          |
| B        | Start of the episode.                                               |         |         |          |
| C        | The agent moves along the top of the level, arriving from the left. |         |         |          |
| D        | The opponent loses by running into a trail.                         |         |         |          |
| Layer 7b | $K = 6$                                                             | CU 0.67 | AS 0.71 | ACU 0.38 |
| E        | The player moves in the upper-left corner.                          |         |         |          |
| F        | The opponent loses the confrontation.                               |         |         |          |
| G        | The player moves in the lower-left corner.                          |         |         |          |
| H        | The end of the game is played in the lower-left corner.             |         |         |          |
| I        | The player starts at the top left and ends at the bottom right.     |         |         |          |
| J        | Start of the episode.                                               |         |         |          |

Table 2.10: CU, AS, and ACU of the two clusterings analyzed in the **Surround** environment.

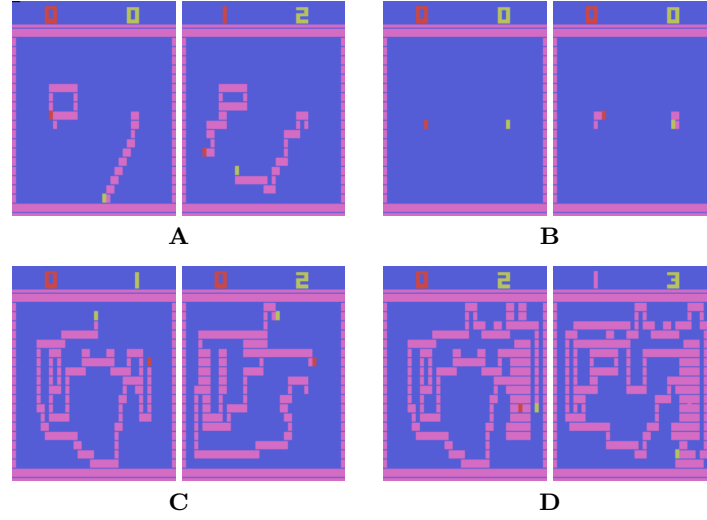


Figure 2.17: Clusters A–D identified by CAR in the **Surround** environment (layer 8b,  $K = 4$ ).

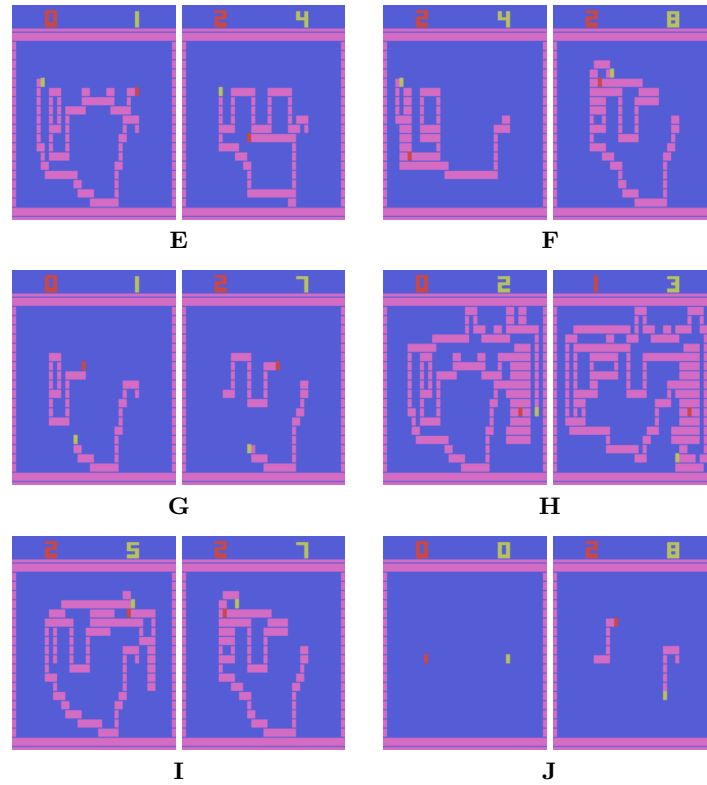


Figure 2.18: Clusters E–J identified by CAR in the **Surround** environment (layer 7b,  $K = 6$ ).

## 2.6 Conclusion

The quantitative results show that CAR consistently recovers similar clustering structures across independently trained surrogate policies. Across the six environments, the CU frequently exceeds 0.5, indicating that independently trained policies can develop similarly structured LRs [Amezquita et al., 2020]. The CU also tends to decrease as the network gets deeper, with the highest values generally observed in the earliest layers. Considering consistency alone would therefore favor the earliest layers of the network.

To complement this analysis, the level of abstraction of these representations must also be considered. For this purpose, AS is analyzed and shows that the earliest layers tend to exhibit lower levels of abstraction. Their representations remain strongly related to the observations and the initial structure of the network. In the deepest layers, the abstraction can decrease again in some environments, as the latent clustering becomes closer to the clustering of the action distribution.

This highlights the importance of considering both consistency and abstraction when identifying meaningful LRs. The ACU addresses this limitation by combining these two properties into a single measure, providing a practical criterion for identifying layers that achieve a suitable balance between representational consistency and abstraction.

These quantitative analyses are complemented by the qualitative analysis, which shows that the clusters identified by CAR correspond to interpretable properties of the environment and the agent’s behavior. Across the different environments, the clusters capture recurring aspects such as episode progression, specific game situations, and behavioral patterns. These observations provide qualitative evidence that the structures recovered by CAR can reflect semantically aspects of the underlying task.

The qualitative analysis also reveals a trade-off associated with the number of clusters. Increasing the number of clusters allows more specific situations to be distinguished, but tends to reduce the stability of the resulting partitions across surrogate policies. Finer clusters have less clear semantic boundaries, reducing agreement across independently trained policies. The choice of the number of clusters therefore involves a trade-off between recovering stable, shared concepts and capturing finer distinctions between situations.

Overall, these results provide empirical support for the assumptions underlying CAR. Independently trained surrogate policies exhibit a degree of convergence in the organization of their LR that cannot be explained solely by the considered baselines. The qualitative analysis also shows that these clusters can correspond to meaningful properties of the environment and the agent’s behavior. Taken together, these findings provide empirical support for the *Representation Convergence Hypothesis* (Hyp. 1).

# Acronyms

|     |                                                                                       |
|-----|---------------------------------------------------------------------------------------|
| 817 | <b>ACU</b> <i>Adjusted Clustering Universality</i> 14, 22–26, 28, 30, 32, 34, 36, 38  |
| 818 | <b>APPO</b> Asynchronous Proximal Policy Optimization 15                              |
| 819 | <b>ARI</b> Adjusted Rand Index 8–12                                                   |
| 820 | <b>AS</b> <i>Abstraction Score</i> 14, 22–26, 28, 30, 32, 34, 36, 38                  |
| 821 | <b>BC</b> Behavioral Cloning 16                                                       |
| 822 | <b>CAR</b> Clustering Agent Representation 1–9, 11, 13–20, 27, 29, 31, 33, 35, 37, 38 |
| 823 | <b>CNN</b> Convolutional Neural Network 19                                            |
| 824 | <b>CS</b> Concept Structure 1–3, 5                                                    |
| 825 | <b>CU</b> <i>Clustering Universality</i> 14, 22–26, 28, 30, 32, 34, 36, 38            |
| 826 | <b>DL</b> Deep Learning 14, 15                                                        |
| 827 | <b>DQN</b> Deep Q-Networks 15                                                         |
| 828 | <b>GPU</b> Graphics Processing Unit 15–17                                             |
| 829 | <b>LR</b> Latent Representation 1–3, 5, 6, 9, 13, 14, 16, 17, 22, 38                  |
| 830 | <b>LS</b> Latent Space 1–9, 11, 12, 22, 25, 26                                        |
| 831 | <b>ME</b> Mechanistic Explainability 1, 2                                             |
| 832 | <b>MLP</b> MultiLayer Perceptron 19                                                   |
| 833 | <b>MSE</b> Mean Squared Error 4–6                                                     |
| 834 | <b>NN</b> Neural Network 1–4, 15, 16                                                  |
| 835 | <b>PPO</b> Proximal Policy Optimization 15, 20, 21                                    |
| 836 | <b>RI</b> Rand Index 10                                                               |
| 837 | <b>RL</b> Reinforcement Learning 1, 4, 13–17                                          |

# Bibliography

- [Amezquita et al., 2020] Amezquita, R. A., Lun, A. T., Becht, E., Carey, V. J., Carpp, L. N., Geistlinger, L., Marini, F., Rue-Albrecht, K., Risso, D., Soneson, C., et al. (2020). Orchestrating single-cell analysis with Bioconductor, volume 17. Nature Publishing Group US New York. **Book**.
- [Apache Arrow, 2016] Apache Arrow (2016). Apache Arrow and Parquet. <https://arrow.apache.org>.
- [Bellemare et al., 2013] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47:253–279. **Article**.
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon, 2019] Falcon, W. (2019). PyTorch Lightning. <https://github.com/Lightning-AI/pytorch-lightning>.
- [Hafner et al., 2023] Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. (2023). Mastering diverse domains through world models.
- [Hubert and Arabie, 1985] Hubert, L. and Arabie, P. (1985). Comparing partitions. Journal of classification, 2(1):193–218. **Article**.
- [Knuth and Moore, 1975] Knuth, D. E. and Moore, R. W. (1975). An analysis of alpha-beta pruning. Artificial intelligence, 6(4):293–326. **Article**.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Machado et al., 2017] Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., and Bowling, M. (2017). Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. **Article. Web**.

- [MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, volume 1, pages 281–297. **Article**.
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.
- [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pages 561–577.
- [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems (NeurIPS).
- [Peyré and Cuturi, 2020] Peyré, G. and Cuturi, M. (2020). Computational optimal transport. **Article**.
- [Rand, 1971] Rand, W. M. (1971). Objective criteria for the evaluation of clustering methods. Journal of the American Statistical association, 66(336):846–850.
- [RAPIDS, 2023] RAPIDS (2023). RAPIDS cuML: GPU-accelerated machine learning. <https://github.com/rapidsai/cuml>.
- [Ross and Bagnell, 2010] Ross, S. and Bagnell, D. (2010). Efficient reductions for imitation learning. In Teh, Y. W. and Titterton, M., editors, Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, volume 9 of Proceedings of Machine Learning Research, pages 661–668, Chia Laguna Resort, Sardinia, Italy. PMLR. **Article**.
- [Ross et al., 2011] Ross, S., Gordon, G., and Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. In Gordon, G., Dunson, D., and Dudík, M., editors, Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, volume 15 of Proceedings of Machine Learning Research, pages 627–635, Fort Lauderdale, FL, USA. PMLR. **Article**.
- [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms.
- [Shlens, 2014] Shlens, J. (2014). Notes on kullback-leibler divergence and likelihood. **Article**.
- [Sutton and Barto, 2018] Sutton, R. S. and Barto, A. G. (2018). Reinforcement Learning: An Introduction. MIT Press. **Book**.

- 910 [Templeton et al., 2024] Templeton, A., Conerly, T., Marcus, J., Lindsey, J.,  
911 Bricken, T., Chen, B., et al. (2024). Scaling monosemanticity: Extracting  
912 interpretable features from claude 3 sonnet. Transformer Circuits Thread.  
913 **Web.**
- 914 [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U.,  
915 De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG,  
916 A., et al. (2024). Gymnasium: A standard interface for reinforcement learning  
917 environments.
- 918 [Zahavy et al., 2017] Zahavy, T., Zrihem, N. B., and Mannor, S. (2017). Graying  
919 the black box: Understanding dqns. **Article.**